

F Y E O

Security Code Review of the Natgold Presale Frontend

Natgold

March 2026
Version 1.0

Presented by:
FYEO Inc.
PO Box 147044
Lakewood CO 80214
United States

Security Level
Public

TABLE OF CONTENTS

Executive Summary.....	2
Overview.....	2
Key Findings.....	2
Scope and Rules of Engagement.....	3
Technical Analyses and Findings.....	6
Findings.....	7
The Classification of vulnerabilities.....	8
Technical Analysis.....	9
Conclusion.....	9
Technical Findings.....	10
General Observations.....	10
Incorrect Gas Cost Validation - Comparing Gas Units to ETH Wei Value.....	11
Sensitive Data Exposure Through Logging and Browser Storage.....	12
Supply Chain Attack Surface.....	13
Large USDT Approval Without Explicit User Consent.....	15
Maintenance Mode Detected But Never Enforced.....	16
Missing Error Handling in KYC Session Key Retrieval.....	17
Missing Request Timeout Configuration.....	18
PAC Expiration Not Validated Client-Side.....	19
Race Condition in Wallet Connection Flow.....	20
Session Token Refresh Failures Not Handled.....	21
Our Process.....	23
Methodology.....	23
Kickoff.....	23
Ramp-up.....	23
Review.....	24
Code Safety.....	24
Technical Specification Matching.....	24
Reporting.....	25
Verify.....	25
Additional Note.....	25

Executive Summary

Overview

Natgold engaged FYEO Inc. to perform a Security Code Review of the Natgold Presale Frontend.

The assessment was conducted remotely by the FYEO Security Team. Testing took place on March 10 - March 13, 2026, and focused on the following objectives:

- To provide the customer with an assessment of their overall security posture and any risks that were discovered within the environment during the engagement.
- To provide a professional opinion on the maturity, adequacy, and efficiency of the security measures that are in place.
- To identify potential issues and include improvement recommendations based on the results of our tests.

This report summarizes the engagement, tests performed, and findings. It also contains detailed descriptions of the discovered vulnerabilities, steps the FYEO Security Team took to identify and validate each issue, as well as any applicable recommendations for remediation.

Key Findings

The following issues have been identified during the testing period. These should be prioritized for remediation to reduce the risk they pose:

- FYEO-NATPFE-01 – Incorrect Gas Cost Validation - Comparing Gas Units to ETH Wei Value
- FYEO-NATPFE-02 – Sensitive Data Exposure Through Logging and Browser Storage
- FYEO-NATPFE-03 – Supply Chain Attack Surface
- FYEO-NATPFE-04 – Large USDT Approval Without Explicit User Consent
- FYEO-NATPFE-05 – Maintenance Mode Detected But Never Enforced
- FYEO-NATPFE-06 – Missing Error Handling in KYC Session Key Retrieval
- FYEO-NATPFE-07 – Missing Request Timeout Configuration
- FYEO-NATPFE-08 – PAC Expiration Not Validated Client-Side
- FYEO-NATPFE-09 – Race Condition in Wallet Connection Flow
- FYEO-NATPFE-10 – Session Token Refresh Failures Not Handled

Based on our review process, we conclude that the reviewed code implements the documented functionality.

Scope and Rules of Engagement

The FYEO Review Team performed a Security Code Review of the Natgold Presale Frontend. The following table documents the targets in scope for the engagement. No additional systems or resources were in scope for this assessment.

The source code was supplied through a private repository at <https://github.com/fpco/natgold-presale-frontend/> with the commit hash 8bb6540428a006f1588c40a4b6fb3c08dcd bdc18.

Remediations were provided with the commit hash a21f2991c3800c20a8ea1c2476ed785a9bf6a7e5.

Files included in the code review	
natgold-presale-frontend/	
├── src/	
│ ├── APIs/	
│ │ ├── Backend.ts	
│ │ ├── Formspree.ts	
│ │ ├── NatGold.ts	
│ │ └── SmartContract.ts	
│ ├── CookieConsent/	
│ │ ├── googleAnalytics.ts	
│ │ ├── types.ts	
│ │ └── useCookieConsentManager.ts	
│ ├── components/	
│ │ ├── CheckoutLayout/	
│ │ │ ├── CheckoutLayout.tsx	
│ │ │ ├── StepClaim.tsx	
│ │ │ ├── StepConfirmation.tsx	
│ │ │ ├── StepGettingStarted.tsx	
│ │ │ ├── StepKYC.tsx	
│ │ │ ├── StepPurchase.tsx	
│ │ │ ├── StepRequirements.tsx	
│ │ │ └── StepTerms.tsx	
│ │ ├── Modals/	
│ │ │ ├── CookieConsent.tsx	
│ │ │ ├── DocumentVerification.tsx	
│ │ │ ├── KycModal.tsx	
│ │ │ ├── Modal.tsx	
│ │ │ ├── ProveOwnershipModal.tsx	
│ │ │ └── PurchaseModal.tsx	

Files included in the code review

- └─ utils.ts
- └─ WelcomeScreen/
 - └─ WelcomeScreen.tsx
- └─ Footer.tsx
- └─ Fund.tsx
- └─ Header.tsx
- └─ MainContent.tsx
- └─ Router.tsx
- └─ RunCrank.tsx
- └─ SendUsdt.tsx
- └─ SignMessage.tsx
- └─ Tools.tsx
- └─ TransitionSettle.tsx
- └─ WalletButton.tsx
- └─ WalletProvider.tsx
- └─ data/
 - └─ Phase.ts
- └─ dataLoaders/
 - └─ useBIV.ts
 - └─ useBalances.ts
 - └─ useChallenge.ts
 - └─ usePacManager.ts
 - └─ usePurchasesForUser.ts
 - └─ useQueryFn.ts
 - └─ useQueryKeyBuilder.ts
 - └─ useSettledAmount.ts
 - └─ useStatus.ts
- └─ redux/
 - └─ slices/
 - └─ appKitConnection.ts
 - └─ authSlice.ts
 - └─ bivSlice.ts
 - └─ ioSlice.ts
 - └─ modalSlice.ts
 - └─ pacSlice.ts
 - └─ phaseSlice.ts
 - └─ routeSlice.ts
 - └─ store.ts
- └─ utils/
 - └─ logging/
 - └─ LogLevel.ts
 - └─ logger.ts
 - └─ HttpIO.ts

Files included in the code review		
		- JsonUtils.ts - bigDecimal.ts - blockchain.ts - decode.ts - number.ts - sessionToken.ts - simulateClaim.ts - time.ts - useAppKitManager.ts - useAuthData.ts - useConnectedAddress.ts - useCurrentStep.ts - useOfflineManager.ts - usePac.ts - useSessionAuthData.ts - useSessionTokenManager.ts
		- App.tsx - LocalStorage.ts - Route.ts - env.ts - main.tsx
		- eslint.config.js - vite.config.ts

Table 1: Scope

Technical Analyses and Findings

During the Security Code Review of the Natgold Presale Frontend, we discovered:

- 3 findings with LOW severity rating.
- 7 findings with INFORMATIONAL severity rating.

The following chart displays the findings by severity.

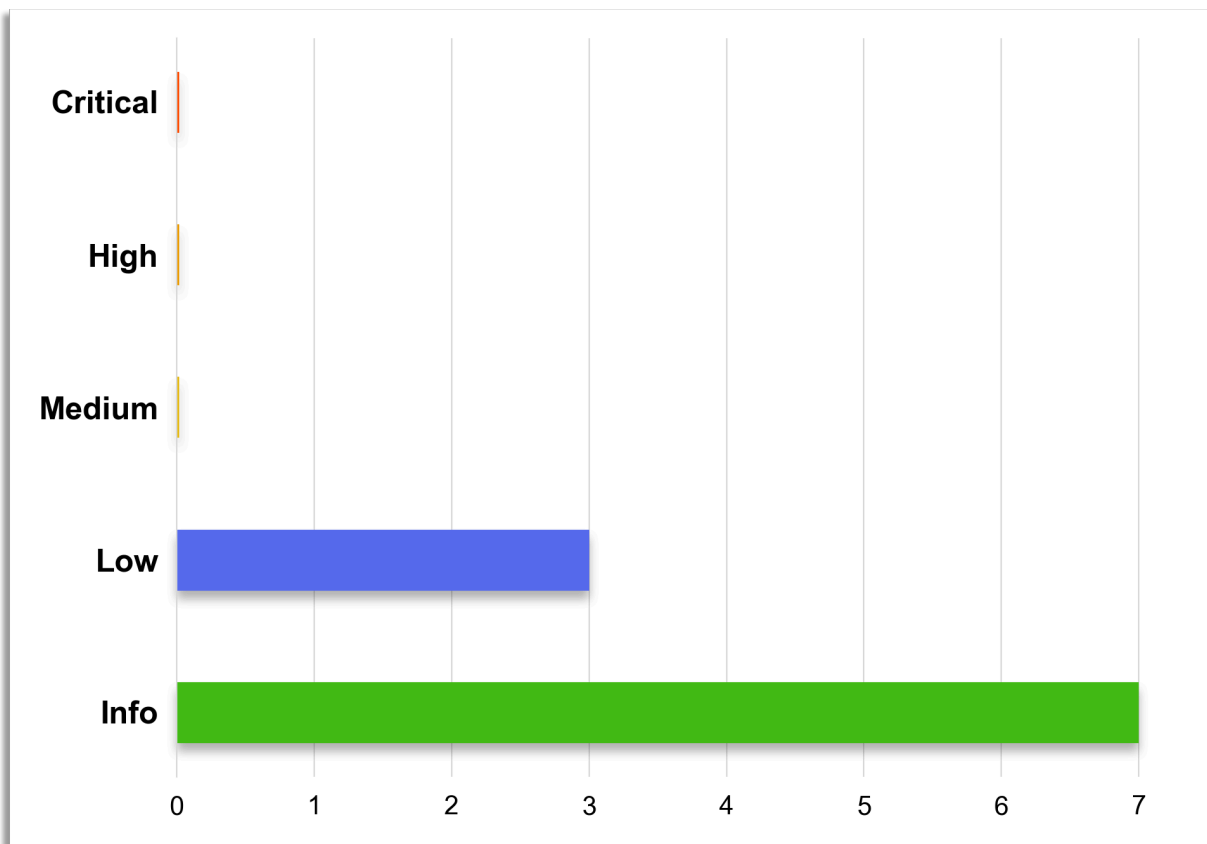


Figure 1: Findings by Severity

Findings

The *Findings* section provides detailed information on each of the findings, including methods of discovery, explanation of severity determination, recommendations, and applicable references.

The following table provides an overview of the findings.

Finding #	Severity	Description	Status
FYEO-NATPFE-01	Low	Incorrect Gas Cost Validation - Comparing Gas Units to ETH Wei Value	Remediated
FYEO-NATPFE-02	Low	Sensitive Data Exposure Through Logging and Browser Storage	Remediated
FYEO-NATPFE-03	Low	Supply Chain Attack Surface	Remediated
FYEO-NATPFE-04	Informational	Large USDT Approval Without Explicit User Consent	Remediated
FYEO-NATPFE-05	Informational	Maintenance Mode Detected But Never Enforced	Acknowledged
FYEO-NATPFE-06	Informational	Missing Error Handling in KYC Session Key Retrieval	Remediated
FYEO-NATPFE-07	Informational	Missing Request Timeout Configuration	Remediated
FYEO-NATPFE-08	Informational	PAC Expiration Not Validated Client-Side	Remediated
FYEO-NATPFE-09	Informational	Race Condition in Wallet Connection Flow	Remediated
FYEO-NATPFE-10	Informational	Session Token Refresh Failures Not Handled	Remediated

--	--	--	--

Table 2: Findings Overview

The Classification of vulnerabilities

Security vulnerabilities and areas for improvement are weighted into one of several categories using, but is not limited to, the criteria listed below:

Critical – vulnerability will lead to a loss of protected assets

- This is a vulnerability that would lead to immediate loss of protected assets
- The complexity to exploit is low
- The probability of exploit is high

High - vulnerability has potential to lead to a loss of protected assets

- All discrepancies found where there is a security claim made in the documentation that cannot be found in the code
- All mismatches from the stated and actual functionality
- Unprotected key material
- Weak encryption of keys
- Badly generated key materials
- Txn signatures not verified
- Spending of funds through logic errors
- Calculation errors overflows and underflows

Medium - vulnerability hampers the uptime of the system or can lead to other problems

- Insecure calls to third party libraries
- Use of untested or nonstandard or non-peer-reviewed crypto functions
- Program crashes, leaves core dumps or writes sensitive data to log files

Low – vulnerability has a security impact but does not directly affect the protected assets

- Overly complex functions
- Unchecked return values from 3rd party libraries that could alter the execution flow

Informational

- General recommendations

Technical Analysis

The source code has been manually validated to the extent that the state of the repository allowed. The validation includes confirming that the code correctly implements the intended functionality.

Conclusion

Based on our review process, we conclude that the code implements the documented functionality to the extent of the reviewed code.

Technical Findings

General Observations

The frontend for the NatGold token presale is a web application that lets users connect their crypto wallet and purchase NatGold tokens using USDT. It walks users through a multi-step checkout flow that starts with connecting a wallet, then moves through identity verification (KYC), accepting terms and conditions, making a purchase, and finally claiming their tokens.

Under the hood, the app manages wallet connections through Reown AppKit, talks to a backend API for authentication and purchase tracking, and interacts directly with a smart contract on-chain for the actual token purchases. Users prove they own their wallet by signing a challenge message, and the app keeps track of their session and purchase authorization codes so they don't have to re-authenticate. There's also a cookie consent system, email notifications after purchases, and a set of debug tools that are only visible on the test environment.

Incorrect Gas Cost Validation - Comparing Gas Units to ETH Wei Value

Finding ID: FYEO-NATPFE-01

Severity: **Low**

Status: **Remediated**

Description

The purchase flow validates whether the user has sufficient ETH to cover transaction costs by comparing gas units directly to the ETH balance in Wei. This comparison is incorrect because gas units (typically 100,000-500,000) represent computational work, while the ETH balance is denominated in Wei (where 1 ETH = 10^{18} Wei). The actual ETH cost should be calculated as gas units multiplied by gas price.

Proof of Issue

File name: src/components/Modals/PurchaseModal.tsx

Line number: 228

```
if (
  simResult.request.gas &&
  simResult.request.gas > balances.eth.value
) {
  toast.error(
    "There was not enough ETH in your wallet to cover transaction costs. Please get more ETH",
  );
  return;
}
```

Severity and Impact Summary

The check will almost always pass incorrectly since Wei values are orders of magnitude larger than gas unit counts. Users could attempt transactions that fail due to insufficient ETH for gas.

Recommendation

Calculate the actual ETH cost by multiplying gas units by the gas price (either from simulation result or current network gas price), then compare this value in Wei against the user's ETH balance.

Sensitive Data Exposure Through Logging and Browser Storage

Finding ID: FYEO-NATPFE-02

Severity: **Low**

Status: **Remediated**

Description

The application logs sensitive data to the browser console and stores authentication tokens in `localStorage` without sanitization. This creates attack vectors for credential theft, particularly once the planned Sentry integration (noted via `TODO`) is implemented.

These areas are affected:

1. `captureNetworkError` in `HttpIO.ts` logs complete request/response details including PII, session tokens, and authorization headers on any API failure
2. The blockchain client in `blockchain.ts` logs all RPC requests and responses, exposing transaction parameters and wallet interactions
3. Session tokens and PAC signatures persist in plaintext `localStorage` (`authSlice.ts`, `pacSlice.ts`)

Proof of Issue

File name: `src/utils/HttpIO.ts`

Line number: 168

The `captureNetworkError` function spreads the full inputs object (including response bodies, URLs, and status codes) into `errorDetails` and logs it as JSON. On KYC-related API failures, this could include names, addresses, and dates of birth.

File name: `src/utils/blockchain.ts`

Line number: 41

The client logs every RPC request and response via `logger.info`, with no filtering or redaction.

File name: `src/redux/slices/authSlice.ts`

Line number: 24

Auth state including session tokens is serialized to `localStorage` in plaintext.

Severity and Impact Summary

Browser console data is accessible to malicious extensions, physical device access, or screen sharing. The `TODO` comment indicates Sentry integration is planned. Without data scrubbing rules, this would transmit sensitive data to a third party. Session tokens in plaintext `localStorage` remain accessible after browser closure, extending the attack window for XSS or physical access scenarios.

Recommendation

Create a logging sanitizer that redacts sensitive fields before logging. Implement log levels to disable verbose logging in production. Before implementing Sentry, establish strict data scrubbing rules. Consider using `sessionStorage` for short-lived tokens.

Supply Chain Attack Surface

Finding ID: FYEO-NATPFE-03

Severity: **Low**

Status: **Remediated**

Description

The build pipeline and HTML entry point have several patterns that widen the supply chain attack surface. While the presale frontend has a limited online lifespan which reduces the practical window for exploitation, these are worth noting.

1. CI uses `npm i` instead of `npm ci`, allowing semver-resolved versions to diverge from the lockfile during builds
2. The Cloudflare build script runs `npm install` before `npm ci`, which can modify the lockfile before the clean install
3. Font Awesome CSS is loaded from cdnjs without a Subresource Integrity (SRI) hash, so a CDN compromise would inject arbitrary CSS into the page
4. The Zendesk widget and Google Analytics scripts are loaded dynamically with no integrity verification and have full DOM access, including to localStorage (session tokens, PAC signatures)

Proof of Issue

File name: .github/workflows/ci.yml

Line number: 26

```
- name: Install  
  run: npm i
```

File name: .github/scripts/cloudflare-build.sh

Line number: 5

```
npm install  
npm ci
```

File name: index.html

Line number: 13

```
<link  
  rel="stylesheet"  
  href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css"  
>
```

File name: index.html

Line number: 27

```
<script id="ze-snippet"  
src="https://static.zdassets.com/ekr/snippet.js?key=25e527da-66cc-4737-ad8f-ac13696480a7"  
> </script>
```

File name: src/CookieConsent/googleAnalytics.ts

Line number: 19

```
const script = document.createElement("script");
script.id = GOOGLE_TAG_SCRIPT_ID;
script.async = true;
script.src = `https://www.googletagmanager.com/gtag/js?id=${measurementId}`;
document.head.appendChild(script);
```

Severity and Impact Summary

The CI issue is the most concrete risk. A compromised patch release of any ^-ranged dependency would be pulled into production builds automatically. The CDN and third-party script concerns are standard vendor trust relationships, but in a wallet-connected application they have access to session tokens and PAC signatures in localStorage. The limited lifespan of the presale significantly reduces the practical exploitation window for all of these.

Recommendation

Replace `npm i` with `npm ci` in the CI workflow and remove the redundant `npm install` from the Cloudflare build script. Add an SRI integrity hash and `crossorigin="anonymous"` to the Font Awesome CDN link.

Large USDT Approval Without Explicit User Consent

Finding ID: FYEO-NATPFE-04

Severity: **Informational**

Status: **Remediated**

Description

The application automatically approves 300,000 USDT to the presale contract without explicitly informing users of the approval amount. While this improves UX by avoiding repeated approvals for multiple purchases, users are not warned about granting such a large token allowance. This violates security best practices around informed consent for token approvals.

Proof of Issue

File name: src/components/Modals/PurchaseModal.tsx

Line number: 25

```
const APPROVE_USDT_AMOUNT = 300000_000000n;

...

await processTransaction(config, async () => {
  return writeContract(config, {
    address: env.usdtAddress,
    abi: erc20Abi,
    functionName: "approve",
    args: [getContract(), APPROVE_USDT_AMOUNT],
  });
});
```

Severity and Impact Summary

Users unknowingly grant large allowances (300,000 USDT). While this is a common UX pattern, the lack of transparency means users cannot make informed decisions about their risk exposure. Security-conscious users may prefer per-transaction approvals or smaller allowance amounts.

Recommendation

Consider adding a user preference option to choose between large one-time approval vs per-transaction approvals.

Maintenance Mode Detected But Never Enforced

Finding ID: FYEO-NATPFE-05

Severity: **Informational**

Status: **Acknowledged**

Description

The `ioSlice` tracks maintenance mode state (`inMaintenance` and `urlsInMaintenance`) and updates it after every API call via `useQueryFn`. However, this state is never read by any component or hook in the codebase. While the system detects when the backend returns 5xx errors, no UI or logic prevents users from taking actions during maintenance.

This creates a gap where users can initiate purchases, complete on-chain transactions, and approve USDT transfers while the backend is degraded. The smart contract calls may succeed, but dependent operations (post-purchase emails, PAC refresh, status updates) will silently fail without user visibility.

Proof of Issue

File name: `src/redux/slices/ioSlice.ts`

Line number: 10

```
type State = {  
  urlsInMaintenance: Record<string, boolean>;  
  inMaintenance: boolean;  
  offline: boolean;  
  sequence: number;  
};
```

Severity and Impact Summary

Users can complete purchases during backend outages, resulting in successful on-chain transactions but failed post-purchase workflows. Session token refresh continues backend requests with no backoff. No user feedback about system degradation.

Recommendation

Add UI indicators when `inMaintenance` is true. Gate purchase flow and other critical operations based on maintenance state. Consider adding maintenance mode checks to data loaders to prevent unnecessary API calls.

Missing Error Handling in KYC Session Key Retrieval

Finding ID: FYEO-NATPFE-06

Severity: **Informational**

Status: **Remediated**

Description

The `getKycSessionKey()` call in the KYC modal's `useEffect` lacks error handling. If the API call fails, the `sardineSessionKey` state remains `null`, which silently disables the Continue button on the address form without providing any feedback to the user about what went wrong.

Proof of Issue

File name: `src/components/Modals/KycModal.tsx`

Line number: 159

```
useEffect(() => {  
  if (!accessToken) {  
    return;  
  }  
  
  getKycSessionKey(accessToken).then((getSessionKeyResult) => {  
    // sardineContext.updateSardineConfig({  
    //   customerId: user?.id?.toString(),  
    //   sessionKey,  
    //   flow: "onboarding",  
    // });  
  
    if (getSessionKeyResult.type === "success") {  
      setSardineSessionKey(getSessionKeyResult.data.session_key);  
    }  
  });  
}, [accessToken]);
```

Severity and Impact Summary

Users who encounter a session key retrieval failure will be unable to proceed with KYC verification and will receive no indication of why the Continue button remains disabled. This creates poor UX and may lead to abandoned KYC attempts.

Recommendation

Add a `.catch()` handler to display an error toast when the session key retrieval fails, allowing users to understand the issue and potentially retry.

Missing Request Timeout Configuration

Finding ID: FYEO-NATPFE-07

Severity: **Informational**

Status: **Remediated**

Description

All API calls via `sendAndCheckWithReporting` lack timeout configuration for fetch requests. This can lead to indefinite hanging requests, degraded user experience, and potential resource exhaustion if the backend becomes unresponsive or network conditions deteriorate.

Proof of Issue

File name: `src/APIs/Backend.ts`

Line number: N/A (affects all API calls)

Severity and Impact Summary

Not a direct security vulnerability, but it could cause poor user experience with frozen UI states and difficulty in error handling. In production environments with unreliable networks or backend issues, users may experience indefinite loading states with no feedback.

Recommendation

Implement timeout handling with appropriate timeout values (30 seconds for standard requests, longer timeouts for document uploads). Add proper error handling and user feedback for timeout scenarios.

PAC Expiration Not Validated Client-Side

Finding ID: FYEO-NATPFE-08

Severity: **Informational**

Status: **Remediated**

Description

The application does not validate PAC expiration at two points: when determining checkout step progression, and when retrieving PAC data for transaction execution. This allows users with expired PACs to progress through the UI and attempt purchases with expired signatures that will fail at the smart contract level. While `usePacManager` has automatic refresh logic that triggers within 10 minutes of expiry, there's a race condition window where expired PACs can be used before refresh completes.

Proof of Issue

File name: `src/utls/useCurrentStep.ts`

Line number: 53

```
if (!pac) {  
  return { maxUnlockedStep, completed };  
}  
  
// Passed KYC  
completed.add(CheckoutStep.KYC);
```

File name: `src/utls/usePac.ts`

Line number: 4

```
const usePac = () => {  
  const connectedAddress = useConnectedAddress();  
  const pac = useAppSelector((s) => s.pac.pac);  
  
  return connectedAddress ? pac[connectedAddress.address] : null;  
};
```

Severity and Impact Summary

The PAC object contains an `expires_at` field, but neither `useCurrentStep` nor `usePac` validate expiration. This creates two problems: Users with expired PACs can reach the PURCHASE step in the UI, and the `usePac` hook returns expired PACs that get used directly in purchase transactions. Failed transactions waste gas fees and create poor user experience with contract revert errors.

Recommendation

Add client-side expiration validation at both locations: In `useCurrentStep`, check both PAC existence and expiration before unlocking subsequent checkout steps. In `usePac`, return null if PAC is expired or expires within a safety buffer to account for transaction processing time and avoid race conditions.

Race Condition in Wallet Connection Flow

Finding ID: FYEO-NATPFE-09

Severity: **Informational**

Status: **Remediated**

Description

The `useAppKitManager` hook incorrectly initializes the `openProveOwnershipRef` flag by checking if the entire PAC dictionary is null, rather than checking if the currently connected wallet has a PAC. The PAC selector `useAppSelector((s) => s.pac.pac)` returns a `Record<string, PacResponse>` object (initially `{}`), which is never null. This causes the flag to always initialize as `false`, preventing the `ProveOwnership` modal from automatically opening when a user connects a wallet for the first time.

The second `useEffect` attempts to correct this by checking `hasPac === false`, but `hasPac` is derived from `pac !== null`, which evaluates to `true` when `pac` is an empty dictionary. This creates a timing window where the `CONNECT_SUCCESS` event could fire before the flag is properly updated, leaving users in a connected but unauthenticated state.

Proof of Issue

File name: `src/utls/useAppKitManager.ts`

Line number: 15

```
const pac = useAppSelector((s) => s.pac.pac);  
const openProveOwnershipRef = useRef(pac === null);
```

Severity and Impact Summary

This prevents the authentication flow from completing automatically for first-time wallet connections, requiring users to manually trigger the `ProveOwnership` modal. While the application functions, it could create poor user experience and could leave users confused about why they cannot proceed with purchases.

Recommendation

Replace the PAC dictionary selector with the `usePac()` hook, which correctly returns null when the currently connected wallet has no PAC. This ensures the flag is properly initialized based on the authentication state of the specific wallet being connected.

Session Token Refresh Failures Not Handled

Finding ID: FYEO-NATPFE-10

Severity: **Informational**

Status: **Remediated**

Description

When `refreshSessionToken()` fails due to network errors, backend errors (401, 500, ...), or other issues, the `useSessionTokenManager` hook silently ignores the failure and leaves the expired or expiring session token in both Redux state and `localStorage`. The application continues attempting to use the invalid token for subsequent API calls, causing silent auth failures until the user manually disconnects and reconnects their wallet.

Proof of Issue

File name: `src/utlis/useSessionTokenManager.ts`

Line number: 21

```
const refreshNow = useCallback(async () => {
  if (!sessionToken || !walletAddress || offline) {
    return;
  }

  const refreshResult = await refreshSessionToken(sessionToken);

  if (refreshResult.type === "success") {
    dispatch(
      updateSessionToken({
        wallet: walletAddress,
        data: refreshResult.data,
      })
    );
  }
  // No handling for failure
}, ...);
```

Severity and Impact Summary

When token refresh fails, all authenticated API calls will fail with 401/403 errors, causing features requiring authentication to silently fail. User experience degrades as there is no feedback about the authentication state, and users must manually disconnect and reconnect their wallet to recover. Common failure scenarios include backend downtime or network interruptions.

The issue is amplified by a race condition between `useSessionTokenManager` (refreshes 5 minutes before expiry) and `useSessionAuthData` (considers tokens expired 30 seconds before expiry). When refresh fails silently, there's a ~4.5-minute window where the refresh has failed but the token hasn't reached the 30-second expiry threshold. `useSessionAuthData` continues returning the expired token as valid. All authenticated API calls fail with 401 errors. The UI shows the user as authenticated but all operations silently fail.

This creates a broken auth state where users appear logged in but cannot perform any authenticated actions.

Recommendation

Handle all failure cases from `refreshSessionToken()` by clearing the expired session token from Redux state and `localStorage` to force a re-authentication flow. Consider implementing retry logic for transient failures, and provide user feedback when the authentication state becomes invalid.

Our Process

Methodology

FYEO Inc. uses the following high-level methodology when approaching engagements. They are broken up into the following phases.



Figure 2: Methodology Flow

Kickoff

The project is kicked off as the sales process has concluded. We typically set up a kickoff meeting where project stakeholders are gathered to discuss the project as well as the responsibilities of participants. During this meeting we verify the scope of the engagement and discuss the project activities. It's an opportunity for both sides to ask questions and get to know each other. By the end of the kickoff there is an understanding of the following:

- Designated points of contact
- Communication methods and frequency
- Shared documentation
- Code and/or any other artifacts necessary for project success
- Follow-up meeting schedule, such as a technical walkthrough
- Understanding of timeline and duration

Ramp-up

Ramp-up consists of the activities necessary to gain proficiency on the project. This can include the steps needed for familiarity with the codebase or technological innovation utilized. This may include, but is not limited to:

- Reviewing previous work in the area including academic papers
- Reviewing programming language constructs for specific languages
- Researching common flaws and recent technological advancements

Review

The review phase is where most of the work on the engagement is completed. This is the phase where we analyze the project for flaws and issues that impact the security posture. Depending on the project this may include an analysis of the architecture, a review of the code, and a specification matching to match the architecture to the implemented code.

In this code audit, we performed the following tasks:

1. Security analysis and architecture review of the original protocol
2. Review of the code written for the project
3. Compliance of the code with the provided technical documentation

The review for this project was performed using manual methods and utilizing the experience of the reviewer. No dynamic testing was performed, only the use of custom-built scripts and tools were used to assist the reviewer during the testing. We discuss our methodology in more detail in the following sections.

Code Safety

We analyzed the provided code, checking for issues related to the following categories:

- General code safety and susceptibility to known issues
- Poor coding practices and unsafe behavior
- Leakage of secrets or other sensitive data through memory mismanagement
- Susceptibility to misuse and system errors
- Error management and logging

This list is general and not comprehensive, meant only to give an understanding of the issues we are looking for.

Technical Specification Matching

We analyzed the provided documentation and checked that the code matches the specification. We checked for things such as:

- Proper implementation of the documented protocol phases
- Proper error handling
- Adherence to the protocol logical description

Reporting

FYEO Inc. delivers a draft report that contains an executive summary, technical details, and observations about the project.

The executive summary contains an overview of the engagement including the number of findings as well as a statement about our general risk assessment of the project. We may conclude that the overall risk is low but depending on what was assessed we may conclude that more scrutiny of the project is needed.

We report security issues identified, as well as informational findings for improvement, categorized by the following labels:

- Critical
- High
- Medium
- Low
- Informational

The technical details are aimed more at developers, describing the issues, the severity ranking and recommendations for mitigation.

As we perform the audit, we may identify issues that aren't security related, but are general best practices and steps that can be taken to lower the attack surface of the project. We will call those out as we encounter them and as time permits.

As an optional step, we can agree on the creation of a public report that can be shared and distributed with a larger audience.

Verify

After the preliminary findings have been delivered, this could be in the form of the approved communication channel or delivery of the draft report, we will verify any fixes within a window of time specified in the project. After the fixes have been verified, we will change the status of the finding in the report from open to remediated.

The output of this phase will be a final report with any mitigated findings noted.

Additional Note

It is important to note that, although we did our best in our analysis, no code audit or assessment is a guarantee of the absence of flaws. Our effort was constrained by resource and time limits along with the scope of the agreement.

While assessing the severity of the findings, we considered the impact, ease of exploitability, and the probability of attack. This is a solid baseline for severity determination.